

LAMeTA: Intent-Aware Agentic Network Optimization via a Large AI Model-Empowered Two-Stage Approach

Yinqiu Liu¹, Member, IEEE, Guangyuan Liu², Graduate Student Member, IEEE,
 Jiacheng Wang¹, Member, IEEE, Ruichen Zhang¹, Member, IEEE, Dusit Niyato³, Fellow, IEEE,
 Geng Sun¹, Senior Member, IEEE, Zehui Xiong⁴, Senior Member, IEEE, and Zhu Han⁵, Fellow, IEEE

Abstract—Nowadays, Generative AI (GenAI) reshapes numerous domains by enabling machines to create content across modalities. As GenAI evolves into autonomous agents capable of reasoning, collaboration, and interaction, they are increasingly deployed on network infrastructures to serve humans automatically. This emerging paradigm, known as the agentic network, presents new optimization challenges due to the demand to incorporate subjective intents of human users expressed in natural language. Traditional generic Deep Reinforcement Learning (DRL) struggles to capture intent semantics and adjust policies dynamically, thus leading to suboptimality. In this paper, we present LAMeTA, a Large AI Model (LAM)-empowered Two-stage Approach for intent-aware agentic network optimization. First, we propose Intent-oriented Knowledge Distillation (IoKD), which efficiently distills intent-understanding capabilities from resource-intensive LAMs to lightweight edge LAMs (E-LAMs) to serve end users. Second, we develop Symbiotic Reinforcement Learning (SRL), integrating E-LAMs with a policy-based DRL framework. In SRL, E-LAMs translate natural language user intents into structured preference vectors that guide both state representation and reward design. The DRL, in turn,

optimizes the generative service function chain composition and E-LAM selection based on real-time network conditions, thus optimizing the subjective Quality-of-Experience (QoE). Extensive experiments conducted in an agentic network with 81 agents demonstrate that IoKD reduces mean squared error in intent prediction by up to 22.5%, while SRL outperforms conventional generic DRL by up to 23.5% in maximizing intent-aware QoE.

Index Terms—Agentic network, large AI model, intent, network optimization.

I. INTRODUCTION

GENERATIVE AI (GenAI) has revolutionized the technological landscape, enabling machines to create content across multiple modalities, including text, images, and videos [1]. Moreover, GenAI is rapidly evolving from basic content generation to complex reasoning and decision-making, transforming how machines interact with and serve humans. This evolution has given rise to GenAI agents [2], which represent intelligent systems that integrate generative capabilities with perception, reasoning, and autonomous problem-solving functionalities to accomplish complex tasks with minimal human intervention. Numerous agents collaborate in advanced communication infrastructures to deliver ubiquitous agentic services, forming agentic networks [2], [3]. Recently, agentic networks have attracted great research attention [1], [2], [4]. For instance, Khowaja et al. [2] proposed a mission-critical agentic network by incorporating various agents into 6G communications. Moreover, agentic networks are also reshaping the Metaverse [4], Digital Twin [1], and various other domains requiring intelligent, coordinated decision-making.

To accomplish advanced tasks, service provisioning in agentic networks often involves multiple heterogeneous agents that form a Generative Service Function Chain (GenSFC) [3], [5]. Consequently, dynamically selecting the optimal agents from various candidates to maximize the efficiency of composed GenSFCs becomes a critical concern. Although traditional optimization methods, such as linear programming [6] and Deep Reinforcement Learning (DRL) [5], [7], can be applied, they face a daunting challenge in agentic networks. As shown in Fig. 1, agents are inherently developed to serve humans [2], incorporating diverse context-driven and subjective intent factors that can hardly be represented. Hence, relying solely

Received 11 May 2025; revised 14 October 2025; accepted 29 November 2025. Date of publication 11 December 2025; date of current version 20 February 2026. This work was supported in part by the Seatrium New Energy Laboratory, Singapore Ministry of Education (MOE) Tier 1 under Grant RT5/23 and Grant RG24/24; in part by Nanyang Technological University Centre in Computational Technologies for Finance (NTU-CCTF); in part by the Research Innovation and Enterprise (RIE) 2025 Industry Alignment Fund-Industry Collaboration Projects (IAF-ICP) administered by the Agency for Science, Technology and Research (A*STAR) under Award I2301E0026; in part by NSF under Grant ECCS-2302469; and in part by the Amazon and Japan Science and Technology Agency (JST) Adopting Sustainable Partnerships for Innovative Research Ecosystem (ASPIRE) under Grant JPMJAP2326. (Yinqiu Liu and Guangyuan Liu contributed equally to this work.) (Corresponding author: Jiacheng Wang.)

Yinqiu Liu, Guangyuan Liu, Jiacheng Wang, Ruichen Zhang, and Dusit Niyato are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: yinqiu001@e.ntu.edu.sg; liug0022@e.ntu.edu.sg; jiacheng.wang@ntu.edu.sg; ruichen.zhang@ntu.edu.sg; dniyato@ntu.edu.sg).

Geng Sun is with the College of Computer Science and Technology, Jilin University, Jilin 130012, China, and also with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: sungeng@jlu.edu.cn).

Zehui Xiong is with the School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, BT7 1NN Belfast, U.K. (e-mail: z.xiong@qub.ac.uk).

Zhu Han is with the Department of Electrical and Computer Engineering, University of Houston, Houston, TX 77004 USA, and also with the Department of Computer Science and Engineering, Kyung Hee University, Seoul 446-701, South Korea (e-mail: hanzhu22@gmail.com).

Digital Object Identifier 10.1109/JSAC.2025.3642840

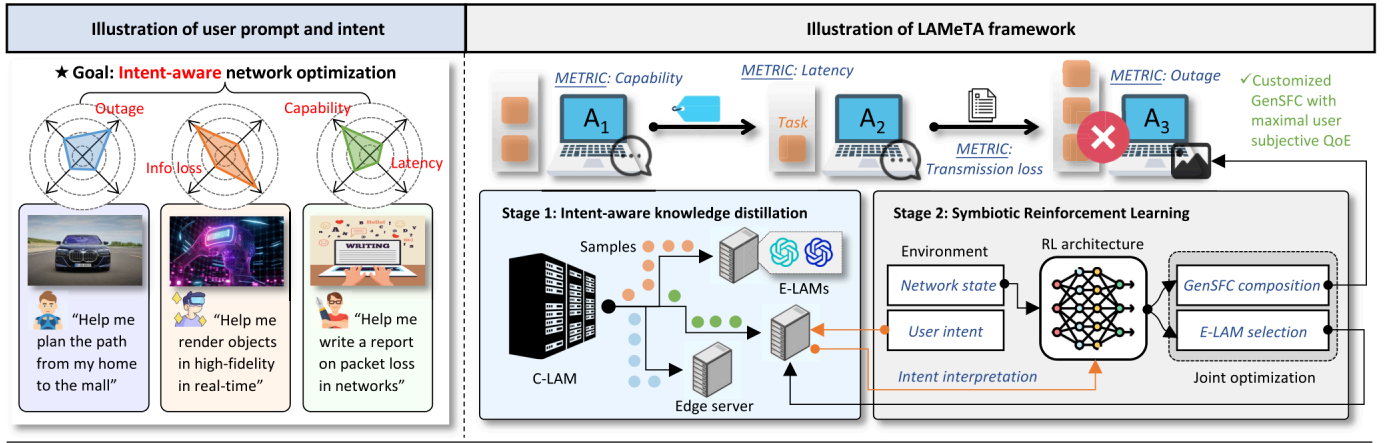


Fig. 1. The illustration of diverse applications and the corresponding pReferences in the agentic network (left). The overall workflow of LAMeTA and the illustration of a 3-step GenSFC for report writing (right). In agentic networks, the intents are incorporated into prompts explicitly or implicitly. For example, example 2 explicitly requires high capability and low latency for 3D rendering. In contrast, the requirement for high capability (to write insightful reports) is implicitly expressed in example 3. Note that the dots of different colors represent user intents from different applications. Orange squares next to agents A_1 , A_2 , and A_3 represent queuing tasks.

on a generic DRL model or static optimization framework is inefficient in satisfying the requirements of diverse users.

Fortunately, Large AI Models (LAMs) have demonstrated remarkable potential in networking optimization [8], [9]. Trained on massive general datasets, LAMs excel in understanding user intent through multimodal comprehension and cross-modality alignment [10]. Moreover, LAMs exhibit strong zero-shot generation capabilities (i.e., directly generate optimization solutions based on user description without retraining), enabling them to tackle unseen tasks with promising performance. For instance, Zhang et al. [8] leveraged a Large Language Model (LLM) to optimize communication systems directly from natural language user descriptions. Shao et al. [11] demonstrated the efficiency of LAMs in spectrum sensing and power allocation optimization. Nonetheless, we notice that LAMs also suffer from two inherent limitations.

- **Strict Resource Requirements:** LAMs significantly exceed conventional AI models in size [2], imposing substantial computing requirements. For instance, GPT-3.5 comprises 175 billion parameters, while current mobile chips, such as *Qualcomm Snapdragon*, can only support LAMs with 10 billion parameters. This contradiction limits the LAM capacity available to end users.
- **Insensitivity to Numerical Conditions:** Most LAMs are optimized for understanding multimedia content, with embeddings and training oriented toward semantic representation [12]. However, solving optimization problems requires precise sensitivity to numerical inputs, as slight changes in environmental factors should be accurately captured by the policy to enable precise decision adjustments. Consequently, although LAMs achieve stratifying zero-shot efficiency, they cannot keep reinforcing the policies based on subtle network conditions.

In this paper, we present LAMeTA, an LAM-empowered Two-stage Approach to perform intent-aware agentic network optimization. First, Intent-oriented Knowledge Distillation (IoKD) addresses resource consumption issues by distilling intent-understanding capability from resource-intensive

LAMs to lightweight edge LAMs (E-LAMs). Then, Symbiotic Reinforcement Learning (SRL) overcomes the numerical insensitivity of LAMs in optimization solving by incorporating a DRL structure. Specifically, we consider agentic network optimization as joint GenSFC composition and E-LAM selection. SRL's policy network continuously interacts with the environment, infers meaningful relationships between changing conditions and optimal solutions, and progressively refines its policy. The selected E-LAM provides intent translation to enhance the state representation and adjust reward settings dynamically. Consequently, the policy network can be reinforced by acquiring reward signals and optimizing the policy gradient [13]. Furthermore, the efficiency of the generated GenSFCs is fed back to the E-LAM, thus achieving continuous self-refinement. In this way, LAMeTA can efficiently provision services in varying agentic networks and maximize users' subjective experience. Notably, our proposal is adaptable to various wireless network optimization tasks that involve complicated human participation, subjective intent factors, and context-specific requirements. Our main contributions can be summarized as follows (see Fig. 1).

- **Intent-aware Agentic Network Optimization:** We present LAMeTA, an LAM-empowered two-stage approach to optimize agentic networks with intents. Particularly, we model subjective Quality-of-Experience (QoE) in agentic networks and formulate a joint optimization problem of GenSFC composition and E-LAM selection.
- **Intent-oriented Knowledge Distillation:** For stage 1, we propose IoKD, enabling lightweight E-LAMs to accurately predict users' subjective preferences based on their intent description. Particularly, to enhance distillation efficiency, IoKD adaptively augments samples that reflect application-specific preference patterns. Moreover, we present a weighted pairwise distillation process, which utilizes contrastive samples and weight adjustment to explicitly guide E-LAMs in distinguishing intent differences and mitigating the impact of outliers.

- **Symbiotic Reinforcement Learning:** For stage 2, we develop SRL, which incorporates an environmental exploration module to interact with the agentic network and policy-based RL to optimize solutions. Moreover, we integrate E-LAMs to translate intents, thus enhancing state representation and adjusting reward functions. The GenSFC performance is then fed back to the E-LAM, realizing symbiotic enhancement.
- **Numerical Results:** We simulate a large agentic network with 81 heterogeneous agents. Extensive experimental results demonstrate that IoKD significantly improves intent understanding precision, improving precision by up to 22.5%. Moreover, SRL consistently outperforms generic DRL methods by 17.2-23.5% in optimizing QoE while adapting to diverse user intents.

The remainder of this paper is structured as follows. First, Section II reviews the related work and states our motivations. The system models and problem formulation are described in Section III. Then, Sections IV and V present IoKD and SRL, respectively. The numerical results are analyzed in Section VI. Finally, Section VII concludes this paper.

II. RELATED WORKS AND MOTIVATION

A. Agentic Networking

Agentic network research has attracted great attention. For instance, Liu et al. [3] presented DyLAN, which introduces different topologies to organize multiple agents and dynamically forms agent teams based on specific user queries to maximize collaboration efficiency. Oriented to 6G networks, Xiao et al. [14] presented AgentNet, a framework that facilitates efficient inter-agent communications and knowledge transfers. Similarly, Dev et al. [15] explored next-generation wireless communications empowered by agents, discussing how to deploy agents in resource-limited and serverless computing networks. Xu et al. [1] introduced a split learning scheme for LLM agents in 6G networks, partitioning agent functions between mobile and edge devices to improve network-based perception and resource efficiency. Khowaja et al. [2] presented a multi-layer agentic AI framework for mission-critical 6G applications, focusing on decentralized decision-making and adaptive resource allocation. Finally, Abdelnabi et al. [16] proposed a firewall-based framework that dynamically secures multi-turn LLM agent communications. Despite such progress, service provisioning in agentic networks remains under-researched. Unlike traditional network scenarios, agents are designed to assist users autonomously, necessitating careful consideration of user intent as a critical factor.

B. LAM-Based Intent Understanding

Trained on vast corpora, LAMs excel in linguistic understanding and generation, allowing users to express complicated intent by natural language from an infinite vocabulary. The authors in [17], [18], and [19] leveraged LAMs to translate user intents into specific network routing, deployment, and function virtualization, respectively. Kou et al. [20] proposed an adaptive LAM, which first comprehends the user intents and

iteratively refines its translation capability from the simulation results. Lira et al. [21] fine-tuned LAM to autonomously convert maintenance intents into comprehensive zero-touch configuration updates about router forwarding tables, dynamic interface reconfigurations, and QoE parameter adjustments. In terms of security, Dzeparoska et al. [22] utilized LLMs to interpret application-layer intents involving the restriction of access to sensitive services and to generate authentication policies accordingly. Finally, LAMs facilitate network optimization by dynamically allocating resources based on user intents. Habib et al. [23] presented an LAM-enhanced resource allocation scheme, in which LAMs are leveraged to classify user intent and feed extracted terms into the DRL. Thus, resource allocations can be optimized by tailoring them to specific user demands. Inspired by such progress, LAMeTA presents IoKD to enhance intent understanding of LAMs and achieves intent-aware optimization.

C. LAM for Network Optimization

1) *Direct Optimization:* LAMs can serve as a powerful zero-shot/few-shot network optimizer [24], [25]. First, optimization problems can be directly expressed through natural language and solved by LAMs given their powerful context understanding capabilities. For instance, the authors in [26], [27], and [28] expressed wireless access point placement, resource allocation, and power control problems by well-crafted prompts, which are fed to LAMs to generate solutions.

2) *Adaptation and Fine-Tuning:* Additionally, LAMs can be refined through domain-specific knowledge adaptation or fine-tuning to further enhance their capabilities. For example, Kan et al. [29] presented MobileLLaMA, an instruction-finetuned LLM that generates codes to optimize packet inspection and IP routing. Shao et al. [11] presented Wireless-LLM, a domain-adapted framework that fuses wireless-specific knowledge within LLMs to optimize power allocation, spectrum sensing, etc. Moreover, LAMs can coordinate multiple tools to optimize network management because their reasoning capabilities enable them to analyze task requirements and automatically schedule specialized tools accordingly. Du et al. [30] presented an LLM-enabled mixture of expert frameworks that dynamically selects specialized DRL models to handle different network optimization tasks. Jiang et al. [31] presented a 6G digital twin network that employs a multi-agent system with planning, reviewing, coding, and execution agents, calling specialized tools, such as cellular traffic load predictors and optimization solvers, to enable autonomous radio management. Huang et al. [32] proposed ChatNet, which adopts four LAMs to perform end-to-end requirement analysis, optimization planning, QoE calculation, and policy execution.

3) *Multimodal Comprehension:* Last but not least, LAMs can process multimodal information that affects optimization objectives, enabling more comprehensive network solutions beyond traditional data types. For instance, Jiang et al. [9] utilized LLMs to extract, align, and recover multimodal semantic information of semantic communications, thus optimizing communication efficiency.

Nonetheless, LAMs cannot efficiently observe the numerical network states and keep reinforcing their policy, which often

leads to suboptimality. To this end, we present SRL, which synergistically combines the semantic understanding capabilities of LAMs with the numerical optimization strengths of DRL architecture.

III. SYSTEM MODEL: AGENTIC NETWORK WITH INTENT-AWARE OPTIMIZATION

In this section, we first model the agentic network environment and QoE factors. Afterward, we formulate the problem of intent-aware genetic network optimization. Finally, we present the architectural design of our LAMeTA.

A. Agentic Network and Services

We consider an agentic network with N GenAI agents $\{A_1, A_2, \dots, A_N\}$, each operating a GenAI model (usually an LAM) to perform a specific type of function. These agents establish connections based on collaborative requirements and underlying network connectivity. Hence, we use an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ to model such a network topology, where the vertices $\mathcal{V}$ and edges $\mathcal{E}$ denote the sets of agents and communication links, respectively.

Users submit service requests to the agentic network. Fig. 1 illustrates an example service request and the corresponding GenSFC. In detail, a user requests the agentic network to write a technical report about packet loss in networks. Accordingly, the GenSFC should comprise three sequential agents: an agent with network access and reasoning capabilities to gather and analyze related documents, an LLM for report writing, and a multimodal LAM with statistical modules to sort data and draw figures and tables. Note that the user prompt implicitly requires maximizing the generation quality. The objective of LAMeTA is to optimize agentic networks to serve users with various requests and intents.

B. Objective QoE Modeling

In this part, we model the QoE of users with the generated GenSFC. Specifically, we involve four factors that are highly oriented to agentic services, namely, agent capability, information loss, service latency, and service outage probability.

1) **Agent Capability:** First, we consider the capability of each agent A_i , $i \in \{1, 2, \dots, N\}$ in GenSFC to perform assigned tasks. Unlike traditional computing architectures [13] where the stakeholders' capability can be reflected directly by the computational power they own, agentic services are based on LAMs, whose resource investment and resulting performance exhibit a complicated pattern. Specifically, the lifecycle of each LAM consists of two distinct stages, namely pre-training and inference.

1) **Pre-training Stage:** During this phase, LAMs learn fundamental patterns, syntactic structures, factual knowledge, and reasoning capabilities from vast and diverse datasets [33]. The ultimate capability of an agent, defined by the generative power of its LAM, is primarily determined by the computational resources and the volume of data invested during this stage. This is because more extensive training with greater compute and larger

datasets enables the LAM to build a more sophisticated understanding of world knowledge.

2) **Inference Stage:** This is the operational phase where a fully trained LAM is deployed and invoked. Based on the specific user prompt, the LAM generates the corresponding output. During the inference stage, the server's computational power primarily affects the latency, or the speed at which a response is generated, rather than the intrinsic quality of the output itself.

Inspired by LAM scaling laws [34], we model the agent capability (determined by the performance of its LAM acquired in the training stage) considering computation budget C_i , available training tokens $|\mathcal{D}_i|$, and the number of parameters $|\mathcal{N}_i|$. Specifically, given C_i , the resulting pre-training loss can be expressed as [34]

$$\mathcal{L}_{\text{pre-training}}^i(|\mathcal{N}_i|, |\mathcal{D}_i|) = \mathcal{L}_0 + \frac{\zeta}{|\mathcal{N}_i|_{\text{OP}}^{\alpha_1}} + \frac{\eta}{|\mathcal{D}_i|_{\text{OP}}^{\alpha_2}}, \quad (1a)$$

$$|\mathcal{N}_i|_{\text{OP}} = \left(\frac{\alpha_1 \zeta}{\alpha_2 \eta} \right)^{\frac{1}{\alpha_1 + \alpha_2}} \left(\frac{C_i}{6} \right)^{\frac{\alpha_2}{\alpha_1 + \alpha_2}}, \quad (1b)$$

$$|\mathcal{D}_i|_{\text{OP}} = \left(\frac{\alpha_2 \eta}{\alpha_1 \zeta} \right)^{\frac{1}{\alpha_1 + \alpha_2}} \left(\frac{C_i}{6} \right)^{\frac{\alpha_1}{\alpha_1 + \alpha_2}}, \quad (1c)$$

where $|\mathcal{N}_i|_{\text{OP}}$ and $|\mathcal{D}_i|_{\text{OP}}$ represent the optimal numbers of LAM parameters and training tokens, respectively. $\mathcal{L}_{\text{pre-training}}^i$ consists of three components: i) $\mathcal{L}_0$ captures the loss of an ideal generative process, which can be regarded as 0, ii) $\frac{\zeta}{|\mathcal{N}_i|_{\text{OP}}^{\alpha_1}}$ captures the fact that an LAM with $|\mathcal{N}_i|_{\text{OP}}$ parameters underperforms the ideal generative process (with infinite parameters), and iii) $\frac{\eta}{|\mathcal{D}_i|_{\text{OP}}^{\alpha_2}}$ captures the fact that the LAM is not trained to convergence, since we only perform a finite number of optimization steps using $|\mathcal{D}_i|_{\text{OP}}$ samples. According to [34], the values of ζ , η , α_1 , and α_2 are empirically set as 406.4, 410.7, 0.34, and 0.28, respectively. With pre-training loss $\mathcal{L}_{\text{pre-training}}^i$, the probability that A_i successfully completes the task can be modeled as $e^{(-\mathcal{L}_{\text{pre-training}}^i)}$ [34]. Consequently, the capability of the entire n -step GenSFC is defined as

$$C_{\text{GenSFC}} = \frac{1}{n} \prod_{i=1}^n \exp(-\mathcal{L}_{\text{pre-training}}^i). \quad (2)$$

Note that agent capability is mainly determined by the LAM training process. In contrast, the computational power assigned for inference primarily affects the processing latency.

2) **Information Loss:** Information loss between agents in a GenSFC is particularly critical, since the prompt for the next agent is the output of the previous agent (see Fig. 1). LAMs are inherently sensitive to prompts since they serve as both instructions and context for generation [9]. Slight errors in prompt transmission can cause significant semantic differences or quality drops. Hence, we derive the total information loss by Bit Error Rate (BER) [35], [36] between adjacent agents in the GenSFC. Suppose that the BER of the communication links connecting agents A_i and A_j is $\mathbb{E}[\text{BER}_j]$, the BER of the entire n -step GenSFC can be expressed as

$$B_{\text{GenSFC}} = 1 - P_{\text{correct}} = 1 - \prod_{i=2}^n (1 - \mathbb{E}[\text{BER}_i]). \quad (3)$$

3) *Service Latency*: Service latency is another critical factor affecting user QoE in agentic networks, as excessive latency significantly degrades user experience. We model each GenSFC as an M/G/1 queuing system [37] and formulate the service latency accordingly. Specifically, we suppose that service requests arrive according to a Poisson process with rate λ . The service time of A_i follows a general distribution with mean $\mathbb{E}[S_i] = \frac{1}{\mu_i}$ and variance σ_i^2 . Each agent provides services in a first-come, first-served manner with a single thread. Finally, the completion of service at agent A_i triggers the arrival of the next agent in the GenSFC.

For a given n -step GenSFC, we analyze end-to-end latency by examining the contribution of each agent to the overall service time. First, the traffic intensity at agent A_i is:

$$\rho_i = \frac{\lambda}{\mu_i}, \rho_i < 1, \quad \forall i \in \{1, 2, \dots, n\}. \quad (4)$$

The traffic intensity represents the ratio between the arrival rate and the service rate, indicating the average utilization of the agent. For system stability across the agentic network, we require $\rho_i < 1$. This constraint ensures that each agent can process requests faster than they arrive, preventing infinite queue buildup. The first moment of service time at agent A_i is $\mathbb{E}[S_i] = \frac{1}{\mu_i}$. The second moment, which captures both the mean and variance of service time, is $\mathbb{E}[S_i^2] = \sigma_i^2 + \left(\frac{1}{\mu_i}\right)^2$. The coefficient of variation, representing the normalized service time variability of agent A_i , is then defined as the ratio of standard deviation to mean:

$$V_i = \sigma_i \mu_i = \frac{\sigma_i}{\mathbb{E}[S_i]}. \quad (5)$$

Using the Pollaczek-Khinchine formula [37], the mean waiting time in queue at agent A_i is:

$$W_i^q = \frac{\lambda \mathbb{E}[S_i^2]}{2(1 - \rho_i)} = \frac{\rho_i(1 + V_i^2)}{2\mu_i(1 - \rho_i)}. \quad (6)$$

Therefore, the mean latency (including both the service and waiting time) at agent A_i is:

$$L_i = \frac{1}{\mu_i} + W_i^q = \frac{1}{\mu_i} \left(1 + \frac{\rho_i(1 + V_i^2)}{2(1 - \rho_i)}\right). \quad (7)$$

In conclusion, the end-to-end service latency across the entire n -step agentic GenSFC is:¹

$$L_{\text{GenSFC}} = \sum_{i=1}^n L_i = \sum_{i=1}^n \frac{1}{\mu_i} \left(1 + \frac{\rho_i(1 + V_i^2)}{2(1 - \rho_i)}\right). \quad (8)$$

4) *Service Outage Probability*: Another issue that affects QoE is the possible service outage of a GenSFC. We consider two primary failure scenarios that may lead to service disruption: i) the crash of an individual agent within the GenSFC, and ii) service denial when the incoming workload exceeds the GenSFC's processing capacity.

¹For agentic AI services, service and waiting time dominate the overall service latency due to the computation-intensive nature of generative tasks. Therefore, we focus on these components and consider the transmission latency to be negligible.

First, the probability that the GenSFC operates normally without any agent outage can be expressed as:

$$P_{\text{no-crash}} = \prod_{i=1}^n \left(1 - \frac{|F_i|}{|O_i|}\right), \quad (9)$$

where $|O_i|$ and $|F_i|$ represent the total number of observations and the number of events that A_i is in outage (caused by hardware failures, software errors, etc.), respectively.

To model the probability of service denial due to excessive workload, we first analyze the maximum sustainable throughput of the GenSFC. Under heavy traffic conditions when $\rho_i \rightarrow 1$ (i.e., $\lambda \rightarrow \mu_i$), we can observe that waiting time increases dramatically, as

$$L_i = \frac{1}{\mu_i} + \frac{\lambda \mathbb{E}[S_i^2]}{2 \left(\frac{\mu_i - \lambda}{\mu_i}\right)} = \frac{1}{\mu_i} + \frac{\lambda \mathbb{E}[S_i^2] \mu_i}{2(\mu_i - \lambda)}. \quad (10)$$

Note that the derivation in (10) is achieved by substituting $1 - \rho_i = 1 - \frac{\lambda}{\mu_i} = \frac{\mu_i - \lambda}{\mu_i}$. Clearly, if $\lambda \rightarrow \min\{\mu_i\}, \forall i \in \{1, 2, \dots, n\}, W_i \rightarrow \infty$. This causes the GenSFC service outage. Hence, to identify the critical bottleneck in the GenSFC, we locate the agent with the highest traffic intensity:

$$i^* = \arg \max_{i \in \{1, 2, \dots, n\}} \{\rho_i\}. \quad (11)$$

The agent A_{i^*} represents the bottleneck in GenSFC and becomes increasingly important as traffic intensifies. Accordingly, the maximum sustainable throughput of the entire GenSFC is constrained by the agent with the lowest service rate:

$$\lambda_{\text{max}} = \mu_{i^*}. \quad (12)$$

(12) defines the fundamental throughput limit of the GenSFC. Attempting to process requests at a rate exceeding λ_{max} will inevitably lead to GenSFC buildup and service outage. Recall that user requests follow a Poisson process with rate λ , the probability of observing more than λ_{max} requests in a unit time interval can be derived as:

$$P_{\text{over}} = P(R > \lambda_{\text{max}}) = 1 - \sum_{k=0}^{\lfloor \lambda_{\text{max}} \rfloor} \frac{e^{-\lambda} \lambda^k}{k!}, \quad (13)$$

where R represents the random variable for the number of arrivals per unit time. Combining both failure scenarios, the probability that the GenSFC operates normally is:

$$\begin{aligned} P_{\text{nor}} &= P_{\text{no-crash}} \cdot (1 - P_{\text{over}}) \\ &= \prod_{i=1}^n \left(1 - \frac{|F_i|}{|O_i|}\right) \left(\sum_{k=0}^{\lfloor \lambda_{\text{max}} \rfloor} \frac{e^{-\lambda} \lambda^k}{k!}\right). \end{aligned} \quad (14)$$

Consequently, the overall probability of service outage $P_{\text{outage}} = 1 - P_{\text{nor}}$. This comprehensive model captures both the reliability aspects of individual agents and the capacity constraints of the overall system, providing a realistic assessment of service availability in agentic networks.

5) *QoE Modeling*: Combining four QoE factors, the overall user QoE toward a given GenSFC can be defined as

$$\begin{aligned} \mathcal{Q}(C_{\text{GenSFC}}, B_{\text{GenSFC}}, W_{\text{GenSFC}}, P_{\text{outage}}) \\ = (1 - P_{\text{outage}}) \left((1 - B_{\text{GenSFC}}) \ln \left(\frac{C_{\text{GenSFC}}}{C_{\text{th}}} \right) \right. \\ \left. - \ln(L_{\text{GenSFC}}) \right), \end{aligned} \quad (15)$$

where C_{th} denotes the user capability threshold. Particularly, to effectively model the user's QoE toward service latency and generation quality, we apply the Weber-Fechner law [38]. This law states that as the stimulus to users (e.g., agent capability) increases, the perceived sensation grows but at a diminishing rate, which can be modeled as a logarithmic relationship.

C. Subjective User Intent and Problem Formulation

Traditionally, the QoE function, such as (15), can be adopted directly as an optimization objective. However, in human-oriented scenarios represented by agentic networks, applying a unified QoE measure is inadequate. This inadequacy stems from the fact that user-specific subjective preferences and task-related intents are overlooked.

We consider the report writing case in Fig. 1 as an example. The user task description and prompt implicitly indicate that C_{GenSFC} and B_{GenSFC} are prioritized over other factors. This preference arises because report writing is particularly sensitive to GenAI capability (to maximize generation quality) and input accuracy (to guarantee that user semantics and generated materials are transmitted along the GenSFC with minimal error). In contrast, for real-time applications, such as 3D environment rendering, service latency becomes the dominant concern. Affected by such task-specific subjective intent, user-perceived QoE differs from the unified QoE defined in (15), which can be expressed as

$$\begin{aligned} \mathcal{Q}(C_{\text{GenSFC}}, B_{\text{GenSFC}}, W_{\text{GenSFC}}, P_{\text{outage}}) \xrightarrow{f_{\theta}(\mathcal{V}^{\infty})=\mathbf{s}} \\ \mathcal{Q}^*(C_{\text{GenSFC}}, B_{\text{GenSFC}}, W_{\text{GenSFC}}, P_{\text{outage}}, \mathbf{s}), \end{aligned} \quad (16)$$

where $f_{\theta}(\cdot)$ denotes an implicit function that maps the objective QoE to user perception based on the specific intent, and $\mathcal{V}^{\infty}$ represents the infinite vocabulary space from which user intents are expressed. The vector $\mathbf{s}$ captures subjective factors generated by $f_{\theta}(\mathcal{V}^{\infty})$. Formulating this transformation faces significant challenges due to two-fold reasons. First, the function $f_{\theta}(\cdot)$ reflects complex cognitive and perceptual processes, making it difficult to formulate or approximate. Second, the input intent encompasses natural language expressions drawn from an infinite vocabulary, creating a vast input space.

To enable intent-aware agentic network optimization, we introduce LAMs to learn and approximate the complex mapping function $f_{\theta}(\mathcal{V}^{\infty})$. As aforementioned, LAMs are particularly well-suited for intent understanding since their extensive pre-training on numerous text corpora enables them to interpret natural language from $\mathcal{V}^{\infty}$. Additionally, LAMs excel at capturing implicit relationships between linguistic expressions and their underlying semantic meanings, making them ideal for modeling the complicated transformation

between user intent and quality perception. Without loss of generality, we define the subjective factor $\mathbf{s}$ as:

$$\mathbf{s} \triangleq [\omega_C, \omega_B, \omega_L, \omega_P], \quad (17)$$

where ω_C , ω_B , ω_L , and ω_P are weighting factors corresponding to capability, information loss, service latency, and service outage probability, respectively. We can observe that user intent affects the weight distribution among different QoE factors, reflecting how users inherently value each aspect of QoE based on their specific applications and preferences. In this case, we further define the intent-aware agentic network optimization problem as

$$\max_{\{\mathcal{G}_{\text{GenSFC}}, \kappa\}} \mathcal{Q}^*(C_{\text{GenSFC}}, B_{\text{GenSFC}}, L_{\text{GenSFC}}, P_{\text{outage}}, \mathbf{s}_{\kappa}) - \mathcal{F}(\kappa), \quad (18a)$$

$$s.t. \quad L_{\text{GenSFC}} \leq L_{\text{max}}, \quad (18b)$$

$$C_{\text{GenSFC}} \geq C_{\text{th}}, \quad (18c)$$

$$\text{Succ}(\mathcal{G}_{\text{GenSFC}}) = \text{True}, \quad (18d)$$

where $\mathcal{G}_{\text{GenSFC}} = (\mathcal{V}_{\text{GenSFC}}, \mathcal{E}_{\text{GenSFC}})$, denoting the generated GenSFC. κ denotes the LAM selected to understand intent, with $\mathbf{s}_{\kappa}$ representing the translated subjective factor. $\mathcal{F}(\kappa)$ represents the service fee for calling LAM κ , which is correlated with the energy consumption of κ . (18b) requires that the service latency cannot exceed the maximum tolerated time. (18c) means that the capability of GenSFC should at least reach the user threshold. Otherwise, the service execution will be considered a failure. (18d) requires that the generated GenSFC should match the user's functional requirements. From (18a), we can observe that we consider a joint optimization problem on both GenSFC composition and LAM selection. Next, we present LAMeTA to solve this problem.

Remark 1: The problem formulated in (18a)-(18d) addresses a core challenge in the practical deployment of agentic networks: achieving effective and personalized multi-agent collaboration. The optimization of the GenSFC composition is fundamental, as the sequential collaboration of heterogeneous agents is a primary mechanism for fulfilling complex, multi-step user requests. Simultaneously, the intertwined problem of E-LAM selection is crucial for translating subjective user intent into an actionable optimization objective. In a realistic ecosystem, there will be a diverse market of E-LAMs, offering different trade-offs between intent-understanding accuracy, resource consumption, and monetary cost.

D. LAMeTA Design Overview

Fig. 2 illustrates the architecture of LAMeTA. A centralized LAM (C-LAM), such as Deepseek R1 or GPT-4 with hundreds of billions of parameters, is deployed by the network infrastructure provider. Serving as a central coordinator, the C-LAM monitors the entire agentic network and exhibits superior comprehension and generation capabilities. However, the influx of users requesting services from the resource-intensive C-LAM often leads to service congestion, increased latency, and potential privacy risks due to the single point of failure. To this end, multiple edge servers are deployed, each of which serves nearby users within a specific application domain.

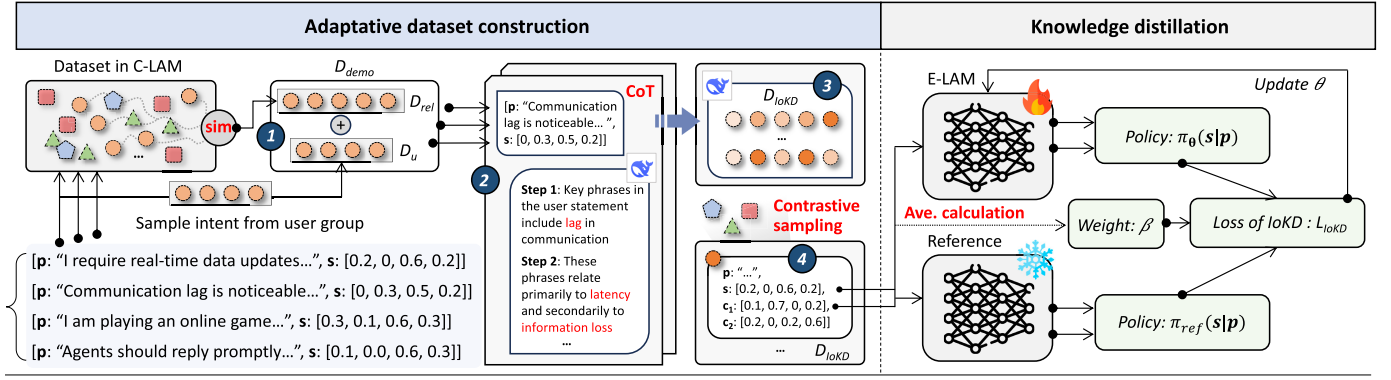


Fig. 3. The illustration of IoKD. The left part demonstrates the process of adaptive dataset construction, which is the prerequisite of IoKD. The right part demonstrates the IoKD process.

the most divergent subjective vectors from $\mathcal{D}_h$. Note that divergence is measured by angular distance, i.e.,

$$\text{Dis}(\mathbf{s}_{\text{IoKD}}^i, \mathbf{s}_h^j) = \frac{\arccos\left(\frac{\mathbf{s}_{\text{IoKD}}^i \cdot \mathbf{s}_h^j}{\|\mathbf{s}_{\text{IoKD}}^i\| \cdot \|\mathbf{s}_h^j\|}\right)}{\pi}, \quad (24)$$

where $\mathbf{s}_{\text{IoKD}}^i = [\omega_C^i, \omega_B^i, \omega_W^i, \omega_P^i]$ is the subjective vector within sample d_{IoKD}^i , $\mathbf{s}_h^j = [\omega_C^j, \omega_B^j, \omega_W^j, \omega_P^j]$ is the subjective vector of the potential contrastive sample $d_h^j \in \mathcal{D}_h$, $\arccos(\cdot)$ is the inverse cosine function³ that converts the cosine similarity to an angle in radians, and $\|\cdot\|$ denotes the Euclidean distance.

For each sample in $\mathcal{D}_{\text{IoKD}}$, we perform top- k selection to identify the most effective contrastive examples:

$$\mathcal{C}_{\text{IoKD}}^i = \text{Top-}k\left(\text{Dis}(\mathbf{s}_{\text{IoKD}}^i, \mathbf{s}_h^j), \forall \mathbf{s}_h^j \in \mathcal{D}_h\right), \quad (25)$$

Finally, the well-prepared $\mathcal{D}_{\text{IoKD}}$ dataset can be formatted as

$$\mathcal{D}_{\text{IoKD}} = \{d_{\text{IoKD}}^i = \langle \mathbf{p}_{\text{IoKD}}^i, \mathbf{s}_{\text{IoKD}}^i, \mathcal{C}_{\text{IoKD}}^i \rangle | i \in \{1, \dots, |\mathcal{D}_{\text{IoKD}}|\}\}. \quad (26)$$

The capability of C-LAM to understand user intents is preserved in $\mathcal{D}_{\text{IoKD}}$, and the entire dataset construction process is illustrated in **Algorithm 1**.

Note that the above dataset construction process is adaptive, which aims to overcome the sample insufficiency in practical agentic networks with heterogeneous application domains. When the number of samples within a specific application is sufficient, the C-LAM can adaptively reduce the degree of augmentation, thereby reducing the computing demand and latency. Moreover, after policies targeting all representative applications are well-trained, users do not need to train new policies, but can directly select the most aligned edge server.

B. Knowledge Distillation Process

Traditional knowledge distillation approaches typically enforce the student model to directly mimic the output probability distribution of the teacher model. However, we intend

³We use the inverse cosine function to compute the angular distance. This metric is specifically chosen because a preference vector $\mathbf{s}$ represents a directional profile of user priorities, where the vector's orientation captures the relative trade-offs among different QoE factors [40]. The angular distance effectively measures the divergence in these directional profiles, unlike the Euclidean (L_2) distance, which is sensitive to the vector's magnitude. A small angle implies that the underlying preference trade-offs are very similar, whereas a large angle indicates highly divergent priorities.

Algorithm 1 Adaptive Dataset Construction for IoKD

Require: C-LAM, Historical dataset $\mathcal{D}_h$, User samples $\mathcal{D}_u$, Number of augmented samples N_{aug} , Similarity threshold τ_{min} , Number of contrastive samples k

Ensure: Augmented dataset $\mathcal{D}_{\text{IoKD}}$ with contrastive samples

- 1: Initialize $\mathcal{D}_{\text{rel}} \leftarrow \emptyset$, $\mathcal{D}_{\text{demo}} \leftarrow \emptyset$, $\mathcal{D}_{\text{IoKD}} \leftarrow \emptyset$
- 2: **Step 1: Semantic Similarity Filtering**
- 3: **for** each $d_h^j = \langle \mathbf{p}_h^j, \mathbf{s}_h^j \rangle \in \mathcal{D}_h$ **do**
- 4: **for** each $d_u^i = \langle \mathbf{p}_u^i, \mathbf{s}_u^i \rangle \in \mathcal{D}_u$ **do**
- 5: Compute $\text{sim}(\mathbf{p}_u^i, \mathbf{p}_h^j)$ by // Eq. (20b)
- 6: **end for**
- 7: **if** $\max_i \text{sim}(\mathbf{p}_u^i, \mathbf{p}_h^j) \geq \tau_{\text{min}}$ **then**
- 8: $\mathcal{D}_{\text{rel}} \leftarrow \mathcal{D}_{\text{rel}} \cup \{d_h^j\}$ // Eq. (20a)
- 9: **end if**
- 10: **end for**
- 11: **Step 2: Chain-of-Thought Construction**
- 12: $\mathcal{D}_{\text{demo}} \leftarrow \mathcal{D}_u \cup \mathcal{D}_{\text{rel}}$ // Eq. (21)
- 13: **for** each $d_{\text{demo}}^i = \langle \mathbf{p}_{\text{demo}}^i, \mathbf{s}_{\text{demo}}^i \rangle \in \mathcal{D}_{\text{demo}}$ **do**
- 14: Generate reasoning path: $\mathcal{R}(d_{\text{demo}}^i)$ by Eq. (22)
- 15: Attach reasoning to the sample
- 16: **end for**
- 17: **Step 3: Dataset Augmentation**
- 18: **for** $k = 1$ to N_{aug} **do**
- 19: Use C-LAM with few-shot prompting from $\mathcal{D}_{\text{demo}}$
- 20: Generate: $d_{\text{syn}}^k = \langle \mathbf{p}_{\text{syn}}^k, \mathbf{s}_{\text{syn}}^k \rangle \leftarrow \text{C-LAM}(\mathcal{D}_{\text{demo}})$
- 21: $\mathcal{D}_{\text{IoKD}} \leftarrow \mathcal{D}_{\text{IoKD}} \cup \{d_{\text{syn}}^k\}$
- 22: **end for**
- 23: $\mathcal{D}_{\text{IoKD}} \leftarrow \mathcal{D}_{\text{demo}} \cup \mathcal{D}_{\text{IoKD}}$ // Eq. (23)
- 24: **Step 4: Contrastive Sampling**
- 25: **for** each $d_{\text{IoKD}}^i = \langle \mathbf{p}_{\text{IoKD}}^i, \mathbf{s}_{\text{IoKD}}^i \rangle \in \mathcal{D}_{\text{IoKD}}$ **do**
- 26: Initialize $\mathcal{C}_{\text{IoKD}}^i \leftarrow \emptyset$
- 27: **for** each $\mathbf{s}_h^j \in \mathcal{D}_h$ **do**
- 28: Compute angular distance: $\text{Dis}(\mathbf{s}_{\text{IoKD}}^i, \mathbf{s}_h^j)$ by Eq. (24)
- 29: **end for**
- 30: Select top- k most divergent: $\mathcal{C}_{\text{IoKD}}^i$ by Eq. (25)
- 31: Augment sample: $d_{\text{IoKD}}^i \leftarrow \langle \mathbf{p}_{\text{IoKD}}^i, \mathbf{s}_{\text{IoKD}}^i, \mathcal{C}_{\text{IoKD}}^i \rangle$ // Eq. (26)
- 32: **end for**
- 33: **return** Augmented dataset $\mathcal{D}_{\text{IoKD}}$

to enable E-LAMs to precisely predict user preferences from intents rather than acquiring the full representation capability of the C-LAM. Inspired by Direct Preference Optimization (DPO) [41], IoKD presents a weighted pairwise distillation approach that focuses on subjective vector predictions. Specifically, IoKD fetches samples from $\mathcal{D}_{\text{IoKD}}$, acquiring prompt $\mathbf{p}$, the corresponding subjective factor $\mathbf{s}_p$, and the contrastive factor $\mathbf{s}_c \in \mathcal{C}$. The prediction of E-LAM should align with $\mathbf{s}_p$ rather than $\mathbf{s}_c$. Such a relationship can be expressed using the Bradley-Terry model [41] as follows:

$$P(\mathbf{s}_p \succ \mathbf{s}_c | \mathbf{p}) = \frac{\exp(r(\mathbf{p}, \mathbf{s}_p))}{\exp(r(\mathbf{p}, \mathbf{s}_p)) + \exp(r(\mathbf{p}, \mathbf{s}_c))}, \quad (27a)$$

$$= \sigma(r(\mathbf{p}, \mathbf{s}_p) - r(\mathbf{p}, \mathbf{s}_c)), \quad (27b)$$

where $r(*, \diamond)$ is a reward function quantifying the appropriateness of subjective vector $\diamond$ for prompt $*$, and $\sigma(x) = \frac{1}{1+\exp(-x)}$ is the logistic function.

Directly modeling $r(\cdot, \cdot)$ requires extensive labeled data to approximate the relationship between prompts and subjective vectors. Hence, we establish a relationship between the reward and E-LAM's policy. To do so, we let $\pi_\theta(\hat{\mathbf{s}} | \mathbf{p})$ represent the E-LAM policy with parameters θ , which computes the probability of generating a predicted subjective vector $\hat{\mathbf{s}}$ given the prompt $\mathbf{p}$. We also define a reference policy $\pi_{\text{ref}}(\hat{\mathbf{s}} | \mathbf{p})$, typically initialized from a pre-trained E-LAM. The key insight is that an optimal policy should assign higher probability to more appropriate subjective vectors, which can be formalized as:

$$\pi_\theta(\hat{\mathbf{s}} | \mathbf{p}) \propto \pi_{\text{ref}}(\hat{\mathbf{s}} | \mathbf{p}) \cdot \exp(r(\mathbf{p}, \hat{\mathbf{s}}) / \beta), \quad (28)$$

where β is a temperature parameter controlling how strongly the policy favors higher-reward outputs. Rearranging (28), we can express the reward function in terms of the policies:

$$r(\mathbf{p}, \hat{\mathbf{s}}) = \beta \log \frac{\pi_\theta(\hat{\mathbf{s}} | \mathbf{p})}{\pi_{\text{ref}}(\hat{\mathbf{s}} | \mathbf{p})} + \beta \log Z(\mathbf{p}), \quad (29)$$

where $Z(\mathbf{p})$ means a normalization constant that depends only on the prompt $\mathbf{p}$. When comparing $\mathbf{s}_p$ and $\mathbf{s}_c$ for the same prompt, this constant term cancels out, i.e.,

$$r(\mathbf{p}, \mathbf{s}_p) - r(\mathbf{p}, \mathbf{s}_c) = \beta \log \frac{\pi_\theta(\mathbf{s}_p | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_p | \mathbf{p})} - \beta \log \frac{\pi_\theta(\mathbf{s}_c | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_c | \mathbf{p})}. \quad (30)$$

Using this implicit reward difference, we define the IoKD loss function as the negative log-likelihood of the observed preferences, i.e.,

$$\mathcal{L}_{\text{IoKD}} = -\mathbb{E}_{\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(\mathbf{s}_p | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_p | \mathbf{p})} - \beta \log \frac{\pi_\theta(\mathbf{s}_c | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_c | \mathbf{p})} \right) \right], \quad (31)$$

where π_θ represents the E-LAM policy being optimized.

Then, we develop a weighted approach to adjust the temperature value β based on the distribution characteristics of subjective vectors. Before the first distillation round, we calculate the average subjective vector of the entire dataset, i.e., $\bar{\mathbf{s}} = \frac{1}{|\mathcal{D}_{\text{IoKD}}|} \sum_{i=1}^{|\mathcal{D}_{\text{IoKD}}|} \mathbf{s}_i$. For each training round, the cosine similarity between the current subjective vector $\mathbf{s}_p$ and $\bar{\mathbf{s}}$ is calculated by (24). Then, β is dynamically set as

$$\beta = \beta_{\text{base}} \cdot (1 - \lambda (\text{Dis}(\mathbf{s}_p, \bar{\mathbf{s}}))), \quad (32)$$

where β_{base} is the default value and λ is a scaling factor. This adjustment mechanism is particularly suitable for numerical vector predictions, as it helps the knowledge distillation process focus on application-specific preference patterns while mitigating the influence of outliers. By assigning a larger β to samples that closely align with the application's average preference pattern, we enlarge the learning signal for these representative cases. Hence, E-LAM can effectively capture the central tendency of user preferences in the target application domain. In contrast, potential outliers that deviate significantly from the average receive a lower β , reducing their impact on the overall learning process.

Algorithm 2 Weighted Pairwise Knowledge Distillation

Require: Augmented dataset $\mathcal{D}_{\text{IoKD}}$ from **Algorithm 1**, Pre-trained E-LAM with parameters θ , Training epochs E , Batch size B , Learning rate η , Base temperature β_{base} , Scaling factor λ

Ensure: Distilled E-LAM with optimized parameters θ^*

```

1: Initialization
2: Initialize reference policy:  $\pi_{\text{ref}}(\mathbf{s} | \mathbf{p}) \leftarrow \pi_\theta(\mathbf{s} | \mathbf{p})$ 
3: Compute average subjective vector:  $\bar{\mathbf{s}} \leftarrow \frac{1}{|\mathcal{D}_{\text{IoKD}}|} \sum_{i=1}^{|\mathcal{D}_{\text{IoKD}}|} \mathbf{s}_i$ 
4: Training Loop
5: for epoch  $e = 1$  to  $E$  do
6:   Shuffle  $\mathcal{D}_{\text{IoKD}}$ 
7:   for each mini-batch  $\mathcal{B}$  of size  $B$  sampled from  $\mathcal{D}_{\text{IoKD}}$  do
8:     Initialize batch loss:  $\mathcal{L}_{\text{batch}} \leftarrow 0$ 
9:     for each sample  $\langle \mathbf{p}, \mathbf{s}_p, \mathcal{C} \rangle \in \mathcal{B}$  do
10:      Sample contrastive vector:  $\mathbf{s}_c \sim \mathcal{C}$ 
11:      // Compute dynamic temperature based on distance to average
12:       $\beta \leftarrow \beta_{\text{base}} \cdot (1 - \lambda \cdot \text{Dis}(\mathbf{s}_p, \bar{\mathbf{s}}))$  // Eq. (32)
13:      // Compute policy log-probability ratios
14:       $r_p \leftarrow \log \frac{\pi_\theta(\mathbf{s}_p | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_p | \mathbf{p})}$ 
15:       $r_c \leftarrow \log \frac{\pi_\theta(\mathbf{s}_c | \mathbf{p})}{\pi_{\text{ref}}(\mathbf{s}_c | \mathbf{p})}$ 
16:      // Compute weighted pairwise distillation loss
17:       $\mathcal{L}_{\text{sample}} \leftarrow -\log \sigma(\beta \cdot r_p - \beta \cdot r_c)$  // Eq. (31)
18:       $\mathcal{L}_{\text{batch}} \leftarrow \mathcal{L}_{\text{batch}} + \mathcal{L}_{\text{sample}}$ 
19:   end for
20:   // Update E-LAM parameters via gradient descent
21:    $\mathcal{L}_{\text{batch}} \leftarrow \frac{\mathcal{L}_{\text{batch}}}{B}$ 
22:   Compute gradient:  $g \leftarrow \nabla_\theta \mathcal{L}_{\text{batch}}$ 
23:   Update parameters:  $\theta \leftarrow \theta - \eta \cdot g$ 
24: end for
25: end for
26: return Optimized E-LAM with parameters  $\theta^*$ 

```

With $\mathcal{L}_{\text{IoKD}}$ and β being configured, we elaborate on the IoKD process (see **Algorithm 2**). First, we initialize the E-LAM with a smaller architecture than C-LAM. The reference model π_{ref} is initialized with the same parameters as the E-LAM to establish a stable starting point. During distillation, IoKD samples batches repeatedly from the constructed dataset $\mathcal{D}_{\text{IoKD}}$. For each sample, we compute β and $\mathcal{L}_{\text{IoKD}}$, thus updating the E-LAM parameters. This preference-based

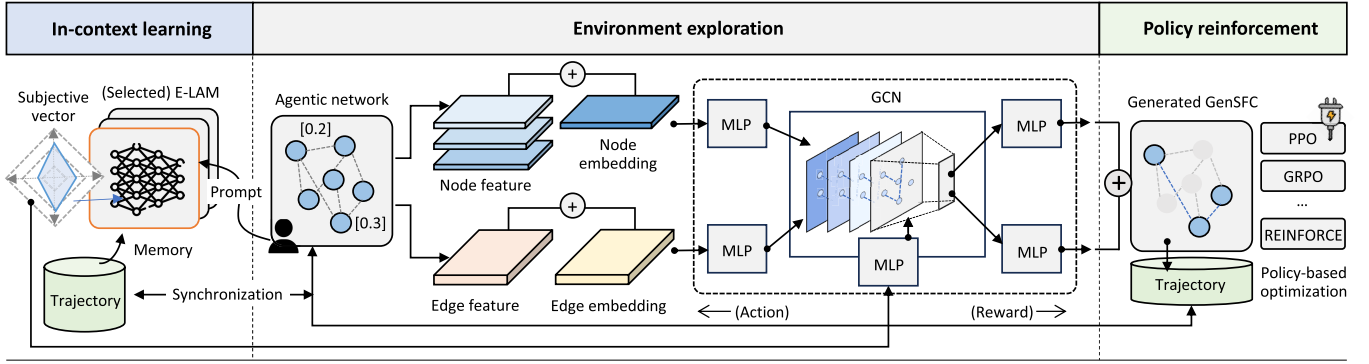


Fig. 4. The illustration of the SRL architecture. Note that the policy optimization module is pluggable and can be implemented by diverse policy-based RL algorithms. Here, we utilize a GCN to learn comprehensive representations of the graph-structured network state, effectively capturing topological relationships between nodes [42]. For the optimization process, we employ PPO due to its excellent balance between sample efficiency and training stability, which is achieved by using a clipped surrogate objective function to prevent destructively large policy updates [13].

learning approach guides the E-LAM in distinguishing appropriate from inappropriate intent mappings while remaining computationally efficient.

V. SYMBIOTIC REINFORCEMENT LEARNING FOR AGENTIC NETWORK OPTIMIZATION

In this section, we present SRL to solve the problem formulated in (18a). First, we introduce SRL's architecture and components. Then, the process of SRL to achieve intent-aware network optimization is discussed, including environment exploration and in-context learning.

A. Learning Architecture Overview

Fig. 4 illustrates the SRL architecture, which includes three modules, i.e., environment exploration, policy optimization, and intent understanding. Specifically, the environment exploration module leverages a Graph Convolutional Network (GCN) [42] to effectively encode the graphical network states. The policy optimization module maintains a policy network that determines the action according to the specific state representation. This module interacts with the agentic network by performing actions and receiving rewards, which indicate the action's desirability. Using gradient-based optimization techniques [13], [43], the policy network progressively learns to map states to high-quality GenSFCs and E-LAM selections that maximize the expected accumulated reward.

Moreover, E-LAM contributes to perceiving the environment. As shown in Fig. 4, it analyzes the user intents expressed in natural language and generates the corresponding subjective vector $\mathbf{s}$. Similar to network state features, $\mathbf{s}$ is also injected into the environment exploration module, forming a comprehensive state representation. Therefore, policy optimization is guided not only by QoE factors but also by subjective intents. Consequently, the generated policies become inherently intent-aware, capable of adapting GenSFC compositions and E-LAM selections to serve specific users. Finally, E-LAM maintains a contextual memory to reinforce itself during training. The overall learning architecture thus exhibits a symbiotic relationship that deeply integrates the intent understanding capabilities of LAMs and numerical optimization of DRL.

B. MDP Design

We formulate the joint optimization problem defined in (18a) as a Markov Decision Process (MDP). Specifically, the key MDP components are defined as follows.

1) *State Space*: Since the agentic network is modeled as an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the state features are associated with both nodes and edges. Moreover, the state space also incorporates the user's subjective preference vector $\mathbf{s}$ predicted by the E-LAM. Consequently, the complete state space is formally defined as:

$$\mathcal{S} \triangleq (\mathcal{S}_V, \mathcal{S}_E, \mathbf{s}), \quad (33a)$$

$$\mathcal{S}_V \triangleq (s_V^1 = \{C_1, L_1, |O_1|, |F_1|\}, s_V^2, \dots, s_V^N), \quad (33b)$$

$$\mathcal{S}_E \triangleq (s_E^{\{1,2\}} = \{\mathbb{E}[BER_2]\}, s_E^{\{1,3\}}, \dots, s_E^{\{N,N-1\}}), \quad (33c)$$

where $\mathcal{S}_V$ and $\mathcal{S}_E$ denote node and edge features, respectively.

2) *Action Space*: The action space $\mathcal{A}$ consists of two components: all candidate nodes (and associated edges) to compose the target GenSFC, and all available E-LAMs for user intent understanding. Note that to generate an n -step GenSFC, each episode is divided into n successive decision steps. At each step, SRL extends the current GenSFC by selecting a new node. Meanwhile, the selected E-LAM translates the user intent into subjective preference vectors that guide the next step. Therefore, each action $a \in \mathcal{A}$ can be represented as $(V_{\text{GenSFC}}, E_{\text{GenSFC}}, \kappa)$, where $V_{\text{GenSFC}} \in \mathcal{V}$ denotes the selected node, E_{GenSFC} is the edge formed by linking it to the predecessor, and κ indicates the selected E-LAM.

3) *Reward Function*: To effectively guide the direction of policy optimization, we design a hierarchical reward function comprising an immediate reward r_{step} and a final performance bonus r_{episode} . At each step, SRL receives r_{step} , which evaluates the desirability of adding a specific node to the GenSFC. Given that agent A_i is selected, r_{step} is defined as:

$$r_{\text{step}} = r_{\text{flag}} + \mathbf{s} \cdot \left[\mathcal{I}(C_i), \mathcal{I}(BER_i), \mathcal{I}(L_i), \mathcal{I}\left(\frac{|F_i|}{|O_i|}\right) \right], \quad (34)$$

where $r_{\text{flag}} = \delta$ if A_i satisfies the functional requirement at this step, and $-\delta$ otherwise. C_i , L_i , and $\frac{|F_i|}{|O_i|}$ represent the capability, service latency, and crash probability of A_i , respectively. BER_i denotes the BER of the communication link between

A_i and its predecessor; for the first node, $BER_i = 0$. $\mathcal{I}(\cdot)$ denotes a normalization function that mitigates the influence of raw value magnitudes to ensure stable learning. By weighting these intent-relevant factors using the predicted user preference vector $\mathbf{s}$, SRL is incentivized to select nodes that better align with subjective service expectations. Upon completion of the full GenSFC, the environment evaluates its overall quality and issues a performance-based bonus r_{episode} . Based on (18a), r_{episode} is defined as in (35), shown at the bottom of the page.

This hierarchical reward structure provides both step-wise guidance and global alignment with user intent, enabling SRL to progressively make informed decisions to maximize user QoE.

C. Environment Exploration and Policy Optimization

We implement the exploration and learning process using a GCN-enhanced PPO framework. In this part, we detail how environmental states are encoded, how actions are selected, and how the policy is optimized.

1) *Graph-Based State Encoding*: Recall that at each step t , the state s_t is formatted as $(\mathcal{S}_V, \mathcal{S}_E, \mathbf{s})$. Let $\mathbf{X} \in \mathbb{R}^{N \times |\mathcal{S}_V|}$ denote the node feature matrix, and $\mathbf{A} \in \mathbb{R}^{N \times N \times |\mathcal{S}_E|}$ denote the binary adjacency matrix. We adopt a multi-layer GCN [42] to embed the spatially-correlated topology. Each GCN layer performs neighborhood aggregation as:

$$\mathbf{H}^{(l+1)} = \sigma \left(\hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (36)$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\hat{\mathbf{D}}$ is the degree matrix of $\hat{\mathbf{A}}$, $\mathbf{W}^{(l)}$ is a learnable weight matrix, and $\sigma(\cdot)$ is the ReLU activation. The initial layer input is $\mathbf{H}^{(0)} = \mathbf{X}$, and multiple such layers propagate contextual information across graph neighborhoods.

After the final GCN layer, a global mean pooling is applied over node embeddings, i.e.,

$$\mathbf{h}_{\text{graph}} = \frac{1}{N} \sum_{i=1}^N \mathbf{H}_i^{(L)}. \quad (37)$$

$\mathbf{h}_{\text{graph}}$ is concatenated with two auxiliary embeddings: a user intent embedding $\mathbf{h}_{\text{intent}} = \phi(\hat{\mathbf{s}})$ and an E-LAM index embedding $\mathbf{h}_{\text{model}} = \psi(m_t)$. These components form the latent representation $\mathbf{z}_t = [\mathbf{h}_{\text{graph}} \parallel \mathbf{h}_{\text{intent}} \parallel \mathbf{h}_{\text{model}}]$, where $\phi(\cdot)$ and $\psi(\cdot)$ are fully connected layers with non-linear activations.

2) *Policy Inference and Action Selection*: The policy is represented by a stochastic mapping $\pi_\varphi(a_t|s_t)$, parameterized by φ , that assigns a probability distribution over actions given the current state. The latent representation $\mathbf{z}_t$ is processed by the actor network to produce this distribution over the discrete action space $\mathcal{A}$, i.e.,

$$\pi_\varphi(a_t|s_t) = \text{Categorical}(\text{MLP}_{\text{actor}}(\mathbf{z}_t)), \quad (38)$$

where MLP denotes a Multilayer Perceptron module [42] for encoding. The critic network, which shares the encoder, estimates the state value function:

$$V_{\varphi'}(s_t) = \text{MLP}_{\text{critic}}(\mathbf{z}_t). \quad (39)$$

In step t , an action a_t is sampled from π_φ , and the environment returns the reward r_{step} and the next state s_{t+1} .

3) *Policy Gradient and Optimization*: To optimize policy, our objective is to maximize the expected cumulative reward:

$$\mathcal{J}(\varphi) = \mathbb{E}_{\pi_\varphi} \left[\sum_{t=0}^T \gamma^t r_t \right]. \quad (40)$$

Using the policy gradient theorem, the gradient of this objective can be written as:

$$\nabla_\varphi \mathcal{J}(\varphi) = \mathbb{E}_{\pi_\varphi} \left[\nabla_\varphi \log \pi_\varphi(a_t|s_t) \cdot \hat{A}_t \right], \quad (41)$$

where $\hat{A}_t$ denotes the advantage estimate indicating how much better an action is compared to the expected baseline. According to [13], $\hat{A}_t$ is defined as

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_{\varphi'}(s_{t+1}) - V_{\varphi'}(s_t). \quad (42)$$

Finally, we utilize PPO to optimize the policy within a trust region. The clipped surrogate objective is defined as

$$\mathcal{L}^{\text{PPO}}(\varphi) = \mathbb{E}_t \left[\min \left(r_t(\varphi) \hat{A}_t, \text{clip}(r_t(\varphi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (43)$$

where $r_t(\varphi) = \frac{\pi_\varphi(a_t|s_t)}{\pi_{\varphi_{\text{old}}}(a_t|s_t)}$ is the importance sampling ratio, and ϵ is a hyperparameter that controls the clipping range. The policy can then be optimized by gradient ascent using $\mathcal{L}^{\text{PPO}}(\varphi)$. Note that LAMeTA regards this part as a pluggable module. Other policy-based RL algorithms, such as REINFORCE and PPO variants, can also be applied.

D. In-Context Learning of E-LAM

Although E-LAMs provide RL with intent understanding capabilities, RL also enhances E-LAMs through environmental feedback. This symbiotic enhancement is achieved through in-context learning [33], a powerful capability of LAMs that allows them to adapt their behavior based on examples provided in the input prompt, without requiring parameter updates or retraining. Specifically, E-LAM maintains a structured context memory $\mathcal{M}$ containing historical records of prompt, predicted subjective vector, and resulting reward, i.e.,

$$\mathcal{M} = \{m_i = \langle \mathbf{p}_i, \mathbf{s}_i, r_i \rangle\}_{i=1}^M. \quad (44)$$

At each step t with prompt $\mathbf{p}_t$, E-LAM retrieves the k most recent examples from $\mathcal{M}$ and analyzes the consistency of preference vectors and reward trends. If fetched subjective vectors maintain high similarity and rewards exhibit a positive trend, this indicates that SRL is operating within a consistent intent domain and producing increasingly effective solutions. In such cases, E-LAM leverages this pattern consistency to reinforce its prediction confidence, i.e.,

$$s_t = \text{E-LAM}(p_t | \{m_{t-1}, m_{t-2}, \dots, m_{t-k}\}) \quad (45)$$

$$r_{\text{episode}} = (1 - \omega_P P_{\text{outage}}) \left((1 - \omega_B B_{\text{GenSFC}}) \ln \left(\frac{\omega_C C_{\text{GenSFC}}}{C_{\text{th}}} \right) - \ln(\omega_L L_{\text{GenSFC}}) \right) - \mathcal{F}(\kappa). \quad (35)$$

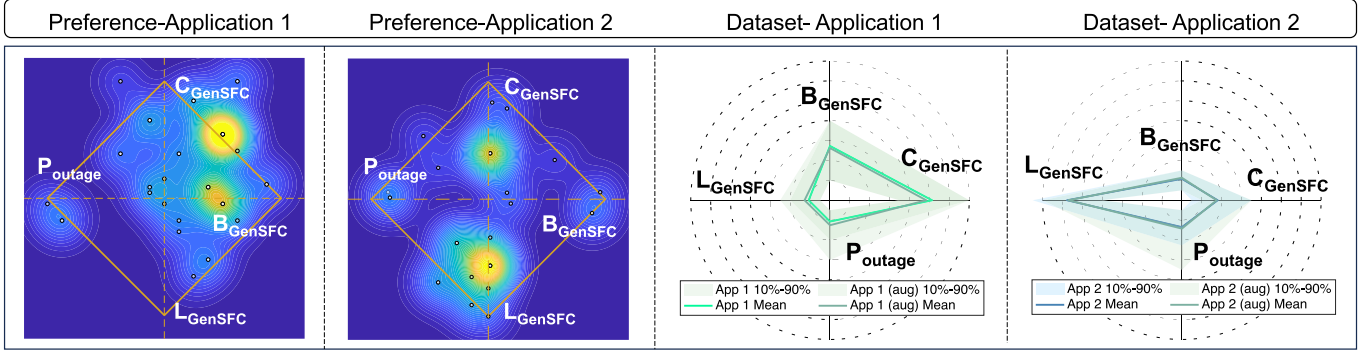


Fig. 5. The illustration of user preferences for two applications and the effectiveness of dataset augmentation.

Conversely, when E-LAM generates a subjective vector that shows greatly different preferences compared with recent examples, it implements an automatic calibration mechanism. This divergence may indicate either an outlier user prompt or a potential misinterpretation. To maintain robust performance, E-LAM employs a weighted averaging approach that moderates potentially outlier translations:

$$s_t^{\text{calibrated}} = \iota \cdot s_t + (1 - \iota) \cdot \bar{s}, \quad (46)$$

where $\bar{s}$ represents the average of recent subjective vectors in $\mathcal{M}$, and $\iota \in [0, 1]$ is a calibration factor that decreases as divergence increases. This calibration ensures that subjective vectors remain within reasonable bounds of the application-specific preference distribution even when facing outliers. Additionally, E-LAM implements a recovery mechanism for cases where it fails to generate valid preference vectors. In such instances, it falls back to $\bar{s}$ for one round.

Through in-context learning, E-LAM achieves adaptive intent understanding without requiring expensive retraining, effectively utilizing the reinforcement learning outcomes to enhance its own capabilities.

The entire procedure of the proposed SRL algorithm is illustrated in **Algorithm 3**.

VI. NUMERICAL RESULTS

In this section, we implement the proposed LAMeTA. Then, we perform extensive experiments to answer 1) whether IoKD can precisely translate user natural language intent into subjective preferences and 2) whether SRL can efficiently compose GenSFCs and select E-LAMs for users according to their specific applications and maximize the QoE.

Testbed: The experiments are conducted on a server with three NVIDIA RTX A5000 GPUs (each has 24 GB of memory) and an AMD Ryzen Threadripper PRO 3975WX 32-core CPU with 263 GB of RAM in Ubuntu 20.04 LTS.

Experimental Settings: We implement LAMeTA and simulate an agentic network consisting of 81 agents, each associated with one of four service types, denoted as type-1 through type-4. In our setup, service requests involve the sequential composition of three service types, i.e., type-1, type-2, and type-3. We apply GPT-4.5⁴ as C-LAM. For E-LAMs,

we apply two variants from Deepseek: a 7-billion-parameter (7B) model and a 1.5-billion-parameter (1.5B) model.⁵

A. Preference Visualization

To investigate how user preferences vary in different applications, we construct a benchmark dataset of 100 human-annotated $\langle \text{prompt}, \text{subjective factor} \rangle$ pairs. The prompts that contain user intent are sampled from two distinct applications: one involving multimedia content generation (Application 1) and another one targeting question-answering (Application 2). We embed the resulting subjective vectors and visualize the aggregated user preferences as heatmaps in Fig. 5. In these heatmaps, the color gradient from black to yellow indicates an increasing degree of user preferences, with yellow hotspots marking the most dominant preference trade-offs.

The visualizations clearly reveal distinct preference profiles for each application. For Application 1, the hotspots are concentrated between the capability (C_{GenSFC}) and information loss (B_{GenSFC}) axes. This distribution visually confirms that users in a multimedia context predominantly prioritize the quality of the generated content and the fidelity of the transmitted information. In contrast, for Application 2, the preference hotspots are located primarily between the latency (L_{GenSFC}) and capability (C_{GenSFC}) axes. This reflects a distinct user demand to receive high-quality answers as rapidly as possible, highlighting a different set of priorities compared to the first application.

For each application, we randomly select 10 samples as few-shot demonstrations and retain the remaining 40 samples as test sets. The C-LAM is then prompted to synthesize $\langle \text{prompt}, \text{subjective factor} \rangle$ pairs, resulting in two augmented datasets with 500 samples. Fig. 5(right) compares the distribution of real and synthesized preference vectors. We can observe that synthetic samples exhibit high semantic similarity to real data, confirming the reliability of C-LAM in preserving task-specific subjective semantics during data augmentation.

B. Precision of User Intent Understanding

We perform IoKD and evaluate the precision of E-LAMs for intent understanding. Specifically, two metrics are used

⁴Model available at: <https://openai.com/index/gpt-4/>

⁵Model available at: <https://huggingface.co/deepseek-ai>

Algorithm 3 Symbiotic Reinforcement Learning (SRL)

Require: Agentic network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, E-LAMs from **Algorithm 2**, User prompt $\mathbf{p}$, Training episodes M , Steps per GenSFC n , Discount factor γ , PPO parameters ϵ, α

Ensure: Optimized policy π_ϕ^* for GenSFC composition and E-LAM selection

- 1: Initialize policy π_ϕ , value network $V_{\phi'}$, E-LAM memory $\mathcal{M} \leftarrow \emptyset$
- 2: Translate intent: $\mathbf{s} \leftarrow \text{E-LAM}(\mathbf{p})$ // Eq. (17)
- 3: **for** episode $m = 1$ to M **do**
- 4: Initialize trajectory $\mathcal{T} \leftarrow \emptyset$, state $s_0 = (\mathcal{S}_V, \mathcal{S}_E, \mathbf{s})$, GenSFC $G_{\text{GenSFC}} \leftarrow \emptyset$
- 5: // *Environment Exploration*
- 6: **for** step $t = 0$ to $n - 1$ **do**
- 7: Encode state: $\mathbf{z}_t = \text{GCN}(\mathcal{S}_V, \mathcal{S}_E) \parallel \varphi(\mathbf{s}) \parallel \psi(\kappa)$ // Eq. (36)-(37)
- 8: Sample action: $a_t = (V, E, \kappa) \sim \pi_\phi(\cdot | s_t)$ // Eq. (38)
- 9: Execute action, compute r_{step} // Eq. (34), observe s_{t+1}
- 10: $\mathcal{T} \leftarrow \mathcal{T} \cup \{(s_t, a_t, r_{\text{step}}, s_{t+1})\}$
- 11: **end for**
- 12: Compute final reward r_{episode} and add to $\mathcal{T}$ // Eq. (35)
- 13: // *Policy Optimization*
- 14: **for each** $(s_t, a_t, r_t, s_{t+1}) \in \mathcal{T}$ **do**
- 15: Compute advantage $\hat{A}_t$ by Eq. (42)
- 16: **end for**
- 17: Update policy: $\phi \leftarrow \phi + \alpha \nabla_\phi \mathbb{E}[\min(r_t(\phi) \hat{A}_t, \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$ // Eq. (43)
- 18: Update value network: $\phi' \leftarrow \phi' + \alpha \nabla_{\phi'} \|V_{\phi'}(s_t) - (r_t + \gamma V_{\phi'}(s_{t+1}))\|^2$
- 19: // *E-LAM In-context Learning*
- 20: $\mathcal{M} \leftarrow \mathcal{M} \cup \{(p, \mathbf{s}, \bar{r}_m)\}$ // Eq. (44)
- 21: **if** divergence detected in recent k records **then**
- 22: Calibrate: $\mathbf{s} \leftarrow \iota \cdot \mathbf{s} + (1 - \iota) \cdot \bar{\mathbf{s}}$ // Eq. (46)
- 23: **end if**
- 24: **end for**
- 25: **return** Optimized policy π_ϕ^*

to measure the prediction error, namely Mean Absolute Error (MAE) and Mean Squared Error (MSE).

- **MAE:** Given n samples, if y_i and $\hat{y}_i$ are the actual and predicted values for the i -th sample, MAE is defined as $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$. MAE's primary focus is on the average error magnitude. It treats all errors equally on a linear scale. Therefore, it is less sensitive to large outlier errors compared to MSE, providing a more robust measure of average performance.
- **MSE:** MSE is calculated as $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$. Due to the squaring term, MSE's focus is on penalizing larger errors more heavily.

In addition to prediction error, we also report the failure rate, defined as the proportion that LAM fails to generate a qualified subjective vector $\mathbf{s}$. As shown in Fig. 6, we begin with a default baseline that applies an even preference vector $\mathbf{s} = [0.25, 0.25, 0.25, 0.25]$ regardless of intent. We can

observe that this setting leads to the highest MAE and MSE across both applications, as it fails to capture any meaningful user-specific variation. Next, we evaluate raw E-LAMs without knowledge distillation. Fig. 6 shows that the 7B variant achieves an MSE of 0.0234 and 0.0282 in Applications 1 and 2, respectively, while the 1.5B variant shows slightly higher error due to its limited capacity. Then, we apply DPO to distill the intent understanding capability from the C-LAM to each E-LAM. The distilled models exhibit substantial improvements: MSE drops to 0.190 for the 7B model and 0.211 for the 1.5B model in Application 1. In addition, failure rates are reduced by more than 50% in most cases. Finally, the proposed IoKD achieves the best performance. By integrating weighted loss, we further reduce MSE by 21.6% (for 7B) and 4.1% (for 1.5B) in Application 1, and 5.1% (for 7B) and 22.5% (for 1.5B) in Application 2, compared with the original DPO. This improvement demonstrates the efficiency of IoKD in guiding E-LAM to learn intent predictions.

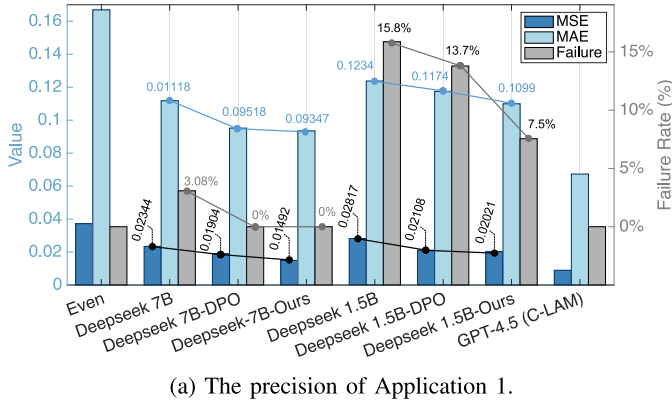
Furthermore, we evaluate the precision of the IoKD with a varying number of samples. As shown in Fig. 7, increasing the number of samples for IoKD keeps increasing the resulting precision, which further demonstrates the effectiveness of our data augmentation algorithm.

C. Efficiency of GenSFC Compositions

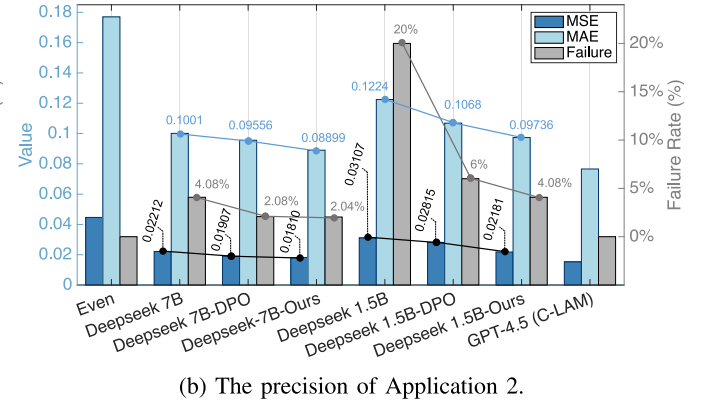
1) *Convergence and Acquired Reward:* We evaluate SRL performance for GenSFC composition and E-LAM selection across the aforementioned applications. As shown in Fig. 8, we compare five baselines: i) **Random**, which composes GenSFCs by randomly selecting agents; ii) **Greedy**, which sequentially selects nodes that maximize the most dominant subjective factor in $\mathbf{s}$; iii) **DRL (Even)**, a PPO-based baseline that assumes even user preferences; iv) **LLM-zero shot**, which queries GPT-4 with the full system model, user intent, and task prompt to generate GenSFCs end-to-end; and v) **SRL**, which integrates intent-aware decision-making.

As shown in Fig. 8(a), Random suffers from severe instability and low utility due to the expansive action space and heterogeneous agent configurations. Similarly, generic DRL, which lacks user-specific guidance, demonstrates inferior performance, as it cannot prioritize the factors that users are concerned about. Although Greedy performs better by aggressively optimizing for the most dominant preference dimension, its unidimensional strategy overlooks trade-offs across other factors, ultimately yielding suboptimal GenSFCs. The LLM zero-shot generation shows inconsistent results. In Application 1, it partially understands the user preference for capability and achieves comparable performance to Greedy. However, in Application 2, it fails to minimize the service latency, leading to significant QoE degradation. In contrast, the proposed SRL consistently outperforms all baselines. It achieves performance improvements of 17.2% and 23.5% over the best-performing alternatives in Applications 1 and 2, respectively. This improvement demonstrates the effectiveness of applying E-LAMs to provide explicit learning signals that guide the policy to align with subjective QoE objectives.

2) *Performance on Each QoE Factor:* To further interpret the intent-aware behavior of SRL, we track the trend of



(a) The precision of Application 1.



(b) The precision of Application 2.

Fig. 6. The precision of user intent translation. Note that the precision is evaluated by averaging the prediction results over 40 test samples for each application. We can observe that IoKD facilitates E-LAMs to approach the performance of C-LAM.

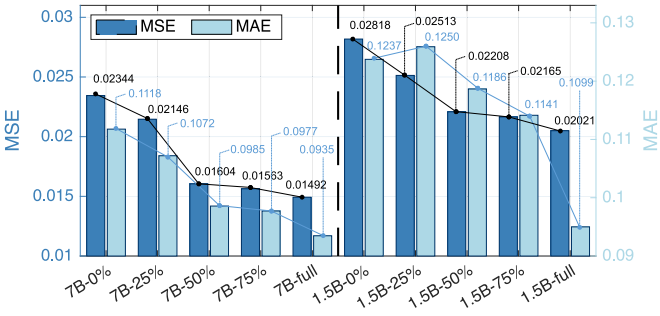


Fig. 7. The impact of dataset sizes for IoKD on intent understanding precision.

each QoE factor during training. As illustrated in Fig. 9, the highlighted segments (marked by red frames) emphasize the critical differences in how our intent-aware SRL and the generic DRL baseline adapt their policies.

In Application 1, where capability and BER are highly weighted, SRL steadily increases C_{GenSFC} while simultaneously reducing B_{GenSFC} . As highlighted by the frame on the ‘Capability’ plot, SRL consistently achieves a higher capability score than the generic DRL baseline. This is because SRL correctly identifies the user’s priority, whereas the conventional DRL learns to wrongly sacrifice this critical factor to optimize a fixed objective. Although P_{outage} remains moderately high, this is an acceptable compromise given its small weight in the user’s preference vector $\mathbf{s}$.

In Application 2, where users prioritize low latency, SRL rapidly reduces L_{GenSFC} by favoring lightweight agents. The frame on the ‘Latency’ plot highlights this success, showing SRL achieving a significantly lower service latency. This latency-driven optimization exemplifies an intelligent trade-off, incurring a certain cost to C_{GenSFC} since lightweight agents often have lower generation capabilities. Nevertheless, because the user’s intent dictates that $\omega_L > \omega_C$, this trade-off is precisely what is desired and leads to a higher subjective QoE. The overall reward remains consistently higher than that of all baselines, confirming that SRL effectively balances competing objectives according to personalized intents.

3) *Intent-Aware GenSFC Composition*: Here, we evaluate the capacity of SRL to customize GenSFCs for users with diverse intents. We simulate 800 users, each associated with a subjective preference vector. For each user, we compute the cosine similarity between its preference vector and the average preference vectors of the two applications. We then apply both trained policies and assess which one achieves a higher reward. As depicted in Fig. 10, over 90% of users can maximize their QoE by selecting the policy trained on the application closest to their personal intent. This alignment confirms that SRL-generated policies encapsulate application-specific patterns. Therefore, LAMeTA allows users to simply select the SRL policy trained by the most similar samples, thus maximizing subjective QoE. This architecture generalizes the agentic network from rigid, fixed-function optimization to flexible, user-driven service provisioning.

4) *Inspection on In-Context Learning*: Fig. 11 demonstrates the validity of in-context learning in SRL. Specifically, we train a policy targeting Application 1 while inserting 20% samples from Application 2 to simulate outliers and failures. We can observe that SRL consistently achieves higher capability scores than the variant without in-context learning, with a final capability of 0.95 compared to 0.91. Moreover, SRL converges more rapidly, reaching the 0.90 threshold approximately 40% faster. This result reveals that outlier detection of in-context learning contributes significantly to preventing error propagation and stabilizing training.

D. Discussion

1) *Adaptability of LAMeTA*: LAMeTA is inherently designed for adaptability and is not constrained to the four factors mentioned in this paper. The adaptability primarily stems from the modularity of our two-stage approach and the powerful zero-shot and few-shot reasoning capabilities of the C-LAM [19]. Should a new application or user group prioritize a different set of factors, such as monetary cost, energy efficiency, or privacy preservation, the framework can be systematically adapted. The process would involve: 1) formally modeling the new performance metrics; 2) rewriting the prompts used for IoKD; and 3) updating the reward function in the SRL agent to align with the new optimization objectives.

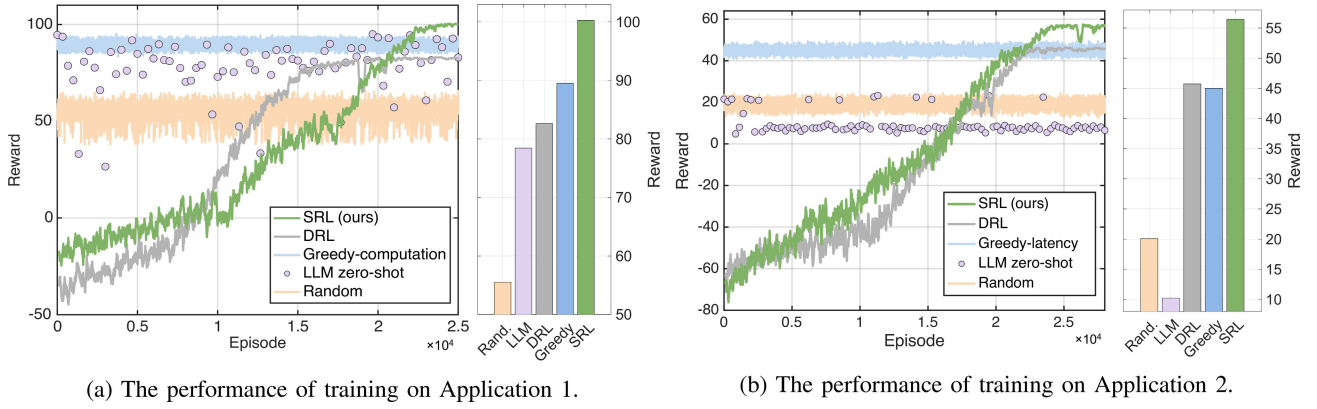


Fig. 8. The convergence and performance of SRL and baselines for agentic network optimization.

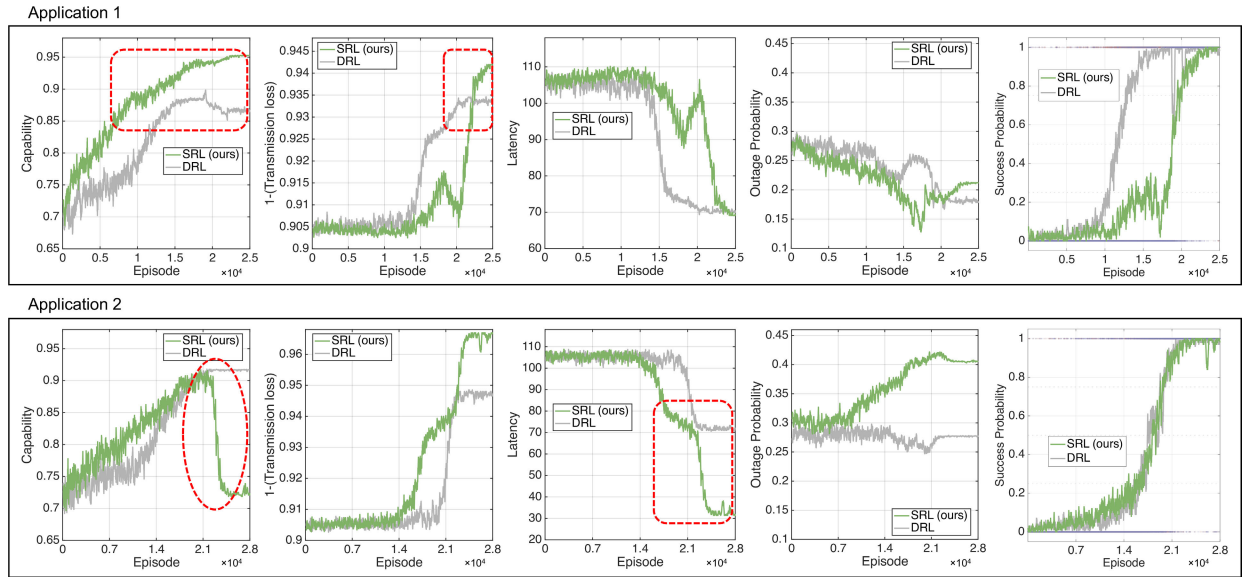


Fig. 9. The trends of each QoE factor during training for Applications 1 (up) and 2 (bottom), respectively.

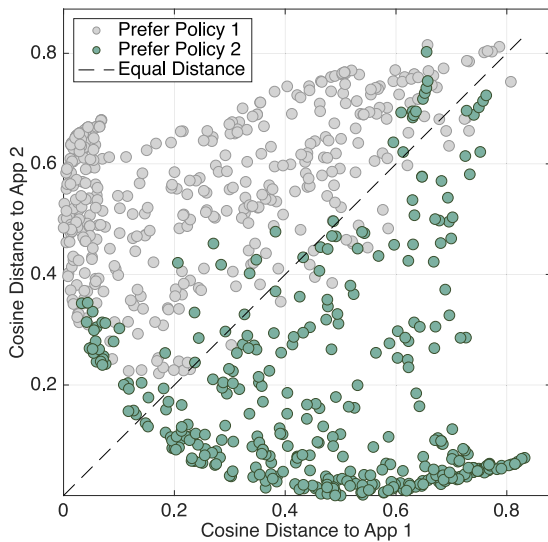


Fig. 10. The illustration of the preferred policy for users with different intents. Note that the distance is measured by cosine similarity and is defined in (24).

Therefore, the core architecture of LAMeTA remains robust, positioning it as a generalizable blueprint for intent-aware

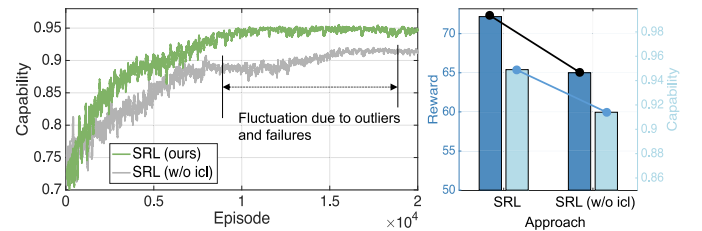


Fig. 11. The ablation study to evaluate the effectiveness of in-context learning.

network optimization across diverse application domains with varying user priorities.

2) *Practical Applications of LAMeTA*: The LAMeTA framework, by enabling the translation of subjective human intent into actionable optimization policies, has significant practical applications across a multitude of domains. We highlight a few key areas below.

- **Intelligent Network Management**: In the evolution towards 6G, network operators can express high-level intents, such as “*Prioritize low-latency and high-bandwidth for streaming services during the live sports*

event.” LAMeTA can translate this complex, multi-objective intent into a concrete resource orchestration policy, dynamically managing network slices and services to satisfy these competing demands in real-time.

- **Complex Enterprise Task Automation:** As illustrated by our report-writing example, LAMeTA can automate sophisticated enterprise workflows. A manager’s request for an insightful sales analysis where “*accuracy is more important than speed*” would be interpreted by our framework to prioritize agents with high capability (C_{GenSFC}). SRL then composes a GenSFC of powerful data analysis and text generation agents to produce a high-quality report that aligns with the user’s strategic goals.
- **Dynamic and Interactive Metaverse:** Agentic networks will be pivotal in creating adaptive virtual worlds. The implicit intent of a player (e.g., requiring a “*challenging puzzle*” over a “*quick combat mission*”) can be captured by LAMeTA. The system will then compose a GenSFC of appropriate narrative, puzzle-generation, and world-building agents on the fly, delivering a truly customized gameplay experience that adapts to subjective desires.

VII. CONCLUSION

In this paper, we have proposed LAMeTA for intent-aware agentic network optimization, designed to bridge the gap between high-level user requirements and low-level network resource allocation. The first stage, IoKD, focuses on creating efficient and specialized agents. We have detailed a process to effectively distill the broad intent-understanding capabilities of C-LAMs into multiple lightweight, user-friendly, and customized E-LAMs. This crucial step enables the deployment of sophisticated intent recognition in resource-constrained edge scenarios, making personalized and scalable agentic services feasible. In the second stage, SRL, we have achieved a seamless integration of perception and decision-making. The E-LAM provides the contextual understanding of user intent, which guides the DRL agent to a more effective and adaptive optimization policy. Our comprehensive experimental results have rigorously validated the effectiveness and superiority of the LAMeTA framework.

REFERENCES

- [1] M. Xu et al., “When large language model agents meet 6G networks: Perception, grounding, and alignment,” *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 63–71, Dec. 2024.
- [2] S. A. Khawaja, K. Dev, M. S. Pathan, E. Zeydan, and M. Debbah, “Integration of agentic AI with 6G networks for mission-critical applications: Use-case and challenges,” 2025, *arXiv:2502.13476*.
- [3] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, “A dynamic LLM-powered agent network for task-oriented agent collaboration,” in *Proc. COLM*, Montreal, QC, Canada, Oct. 2023, pp. 1–30.
- [4] K. Siau and Y. Zhang, “Meta-entrepreneurship: An analysis theory on integrating generative AI, agentic AI, and metaverse for entrepreneurship,” *J. Global Inf. Manage. (JGIM)*, vol. 32, no. 1, pp. 1–21, Dec. 2024.
- [5] Y. Yue et al., “MasRouter: Learning to route LLMs for multi-agent systems,” 2025, *arXiv:2502.11133*.
- [6] B. Jaumard and Q. H. Duong, “A nested decomposition model for reliable NFV 5G network slicing,” *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2186–2200, Sep. 2023.
- [7] T. Dong et al., “Standing on the shoulders of giants: Cross-slice federated meta learning for resource orchestration to cold-start slice,” *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 828–845, Apr. 2023.
- [8] R. Zhang et al., “Generative AI agents with large language model for satellite networks via a mixture of experts transmission,” *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3581–3596, Dec. 2024.
- [9] F. Jiang et al., “Large AI model empowered multimodal semantic communications,” *IEEE Commun. Mag.*, vol. 63, no. 1, pp. 76–82, Jan. 2025.
- [10] F. Jiang et al., “Large AI model-based semantic communications,” *IEEE Wireless Commun.*, vol. 31, no. 3, pp. 68–75, Jun. 2024.
- [11] J. Shao et al., “WirelessLLM: Empowering large language models towards wireless intelligence,” *J. Commun. Inf. Netw.*, vol. 9, no. 2, pp. 99–112, Jun. 2024.
- [12] Y.-C. Chen et al., “UNITER: UNiversal image-TExt representation learning,” in *Proc. ECCV*, Aug. 2020, pp. 104–120.
- [13] R. Zhang, K. Xiong, Y. Lu, P. Fan, D. W. K. Ng, and K. B. Letaief, “Energy efficiency maximization in RIS-assisted SWIPT networks with RSMA: A PPO-based approach,” *IEEE J. Sel. Areas Commun.*, vol. 41, no. 5, pp. 1413–1430, May 2023.
- [14] Y. Xiao, G. Shi, and P. Zhang, “Towards agentic AI networking in 6G: A generative foundation model-as-agent approach,” 2025, *arXiv:2503.15764*.
- [15] K. Dev, S. A. Khawaja, K. Singh, E. Zeydan, and M. Debbah, “Advanced architectures integrated with agentic AI for next-generation wireless networks,” 2025, *arXiv:2502.01089*.
- [16] S. Abdelnabi, A. Gomaa, E. Bagdasarian, P. O. Kristensson, and R. Shokri, “Firewalls to secure dynamic LLM agentic networks,” 2025, *arXiv:2502.01822*.
- [17] A. Mekrache, A. Ksentini, and C. Verikoukis, “Intent-based management of next-generation networks: An LLM-centric approach,” *IEEE Netw.*, vol. 38, no. 5, pp. 29–36, Sep. 2024.
- [18] D. M. Manias, A. Chouman, and A. Shami, “Towards intent-based network management: Large language models for intent extraction in 5G core networks,” in *Proc. 20th Int. Conf. Design Reliable Commun. Netw. (DRCN)*, Montreal, QC, Canada, May 2024, pp. 1–6.
- [19] N. Van Tu, J.-H. Yoo, and J. W.-K. Hong, “Towards intent-based configuration for network function virtualization using in-context learning in large language models,” in *Proc. NOMS IEEE Netw. Oper. Manage. Symp.*, Seoul, South Korea, May 2024, pp. 1–8.
- [20] S. Kou, C. Yang, and M. Gurusamy, “GIA: LLM-enabled generative intent abstraction to enhance adaptability for intent-driven networks,” *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 2, pp. 999–1012, Apr. 2025.
- [21] O. G. Lira, O. M. Caicedo, and N. L. S. da Fonseca, “Large language models for zero touch network configuration management,” *IEEE Commun. Mag.*, vol. 63, no. 7, pp. 146–153, Jul. 2025.
- [22] K. Dzevaroska, J. Lin, A. Tizghadam, and A. Leon-Garcia, “LLM-based policy generation for intent-based management of applications,” in *Proc. 19th Int. Conf. Netw. Service Manage. (CNSM)*, Oct. 2023, pp. 1–7.
- [23] M. A. Habib et al., “LLM-based intent processing and network optimization using attention-based hierarchical reinforcement learning,” in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Milan, Italy, Mar. 2025, pp. 1–6.
- [24] S. Long et al., “A survey on intelligent network operations and performance optimization based on large language models,” *IEEE Commun. Surveys Tuts.*, vol. 27, no. 6, pp. 3915–3949, Dec. 2025.
- [25] H. Zhou et al., “Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities,” *IEEE Commun. Surveys Tuts.*, vol. 27, no. 3, pp. 1955–2005, Jun. 2025.
- [26] K. Qiu, S. Bakirtzis, I. Wassell, H. Song, J. Zhang, and K. Wang, “Large language model-based wireless network design,” *IEEE Wireless Commun. Lett.*, vol. 13, no. 12, pp. 3340–3344, Dec. 2024.
- [27] W. Lee and J. Park, “LLM-empowered resource allocation in wireless communications systems,” 2024, *arXiv:2408.02944*.
- [28] H. Zhou et al., “Large language model (LLM)-enabled in-context learning for wireless network optimization: A case study of power control,” 2024, *arXiv:2408.00214*.
- [29] K. B. Kan, H. Mun, G. Cao, and Y. Lee, “Mobile-LLaMA: Instruction fine-tuning open-source LLM for network analysis in 5G networks,” *IEEE Netw.*, vol. 38, no. 5, pp. 76–83, Sep. 2024.
- [30] H. Du et al., “Mixture of experts for intelligent networks: A large language model-enabled approach,” in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, May 2024, pp. 531–536.

- [31] S. Jiang, B. Lin, Y. Wu, and Y. Gao, "LINKs: Large language model integrated management for 6G empowered digital twin NetwOrKs," in *Proc. IEEE 100th Veh. Technol. Conf. (VTC-Fall)*, Oct. 2024, pp. 1–6.
- [32] Y. Huang et al., "Large language models for networking: Applications, enabling techniques, and challenges," *IEEE Netw.*, vol. 39, no. 1, pp. 235–242, Jan. 2025.
- [33] A. Mohamed, M. E. Rashid, and K. Shaalan, "In-context learning in large language models (LLMs): Mechanisms, capabilities, and implications for advanced knowledge representation and reasoning," *IEEE Access*, vol. 13, pp. 95574–95593, 2025.
- [34] J. Hoffmann et al., "Training compute-optimal large language models," in *Proc. NIPS*, New Orleans, LA, USA, Nov. 2022, pp. 30016–30030.
- [35] M. A. Rahman and H. Harada, "New exact closed-form PDF of the sum of Nakagami- m random variables with applications," *IEEE Trans. Commun.*, vol. 59, no. 2, pp. 395–401, Feb. 2011.
- [36] Z. Ma et al., "Modeling and analysis of MIMO multipath channels with aerial intelligent reflecting surface," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 10, pp. 3027–3040, Oct. 2022.
- [37] J. Wang, "An M/G/1 queue with second optional service and server breakdowns," *Comput. Math. With Appl.*, vol. 47, nos. 10–11, pp. 1713–1723, May 2004.
- [38] H. Du et al., "Attention-aware resource allocation and QoE analysis for metaverse xURLLC services," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 7, pp. 2158–2175, Jul. 2023.
- [39] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, Hong Kong, Nov. 2019, pp. 3980–3990.
- [40] W. Minmin, S. Shengli, and L. Yejin, "A compressive tracking method based on Gaussian differential graph and weighted cosine similarity metric," *IEEE Signal Process. Lett.*, vol. 25, no. 4, pp. 501–505, Apr. 2018.
- [41] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. NIPS*, New Orleans, LA, USA, Dec. 2023, pp. 53728–53741.
- [42] L. Yuan, P. Jiang, W. Hou, and W. Huang, "G-MLP: Graph multi-layer perceptron for node classification using contrastive learning," *IEEE Access*, vol. 12, pp. 104909–104919, 2024.
- [43] Z. Ma et al., "Deep reinforcement learning for energy efficiency maximization in RSMA-IRS-assisted ISAC system," *IEEE Trans. Veh. Technol.*, vol. 74, no. 11, pp. 1–6, Nov. 2025.